Warning_Engine

Warning Engine + Session Orchestration

Role

You are the decision layer. Person 3 tells you what each window contains. You decide what action to take — stay silent, issue a warning, or terminate the session. You also track the full session timeline and build the end-of-session drift summary that Person 1 uses to generate the QA verdict. The entire value of the proctoring system is visible through your outputs.

Data Flow

```
Person 3 ──► POST /window/result    (window classifications, continuously)
                   │
                   ▼
             Update session state
             (consecutive incorrect counter, total windows, warning history)
                   │
                   ▼
             Warning Decision Engine
             (1→ignore, 2→mild, 3-4→strong, 5+→severe)
                   │
                   ├──► POST /warnings/log → Person 1 (relay to frontend)
                   │
                   └──► POST /session/{id}/terminate → Person 1 (if severe)

Person 3 ──► POST /session/{id}/end
                   │
                   ▼
             Build drift summary
                   │
                   ▼
             POST /session/{id}/drift-summary → Person 1
```

What You Build

1. Service Setup — Port 8003

In-memory state per active session:

- `total_windows` — total number of windows received
- `incorrect_windows` — count of windows classified as incorrect or out_of_scope
- `consecutive_incorrect` — current streak of bad windows (resets to 0 on any correct/weakly_correct window)
- `max_consecutive_incorrect` — highest streak seen in this session
- `warning_history` — list of all warnings issued
- `window_log` — all window results received (for summary)
- `terminated` — boolean, true if session was force-closed

---

2. Session Init Endpoint

POST `/session/{barter_id}/init`

Called by Person 1 when the session starts. Creates the empty state structure above.

Response:

```
{ "status": "initialized", "barter_id": 1 }
```

---

3. Window Result Ingest Endpoint

POST `/window/result`

Called by Person 3 for every completed window. Update counters and run the warning decision engine.

Counter update rules:

- If classification is `incorrect` or `out_of_scope` → increment `consecutive_incorrect`, increment `incorrect_windows`
- If classification is `correct` or `weakly_correct` → reset `consecutive_incorrect` to 0
- Always increment `total_windows`

Request (from Person 3):


```json
{
  "barter_id": 1,
  "window_id": 3,
  "classification": "incorrect",
  "similarity_score": 0.2341,
  "scope_violation": false,
  "timestamp_start": 1710234200.0,
  "timestamp_end": 1710234300.0,
  "text_preview": "So let me talk about how the stock market works..."
}
```


Response:


```json
{ "status": "processed", "consecutive_incorrect": 2 }
```


---

4. Warning Decision Engine

Applied after every incoming window result.

Trigger	Severity	Reason shown to users
`consecutive_incorrect == 1`	— (silent)	Nothing shown
`consecutive_incorrect == 2`	mild	"The session appears to be moving off topic. Please refocus."
`consecutive_incorrect == 3 or 4`	strong	"The session has significantly drifted from the agreed topic. A warning has been logged."
`consecutive_incorrect >= 5`	severe	"Severe topic drift detected. The session may be terminated." → also

Trigger	Severity	Reason shown to users
		triggers session termination
`scope_violation == true` (any window)	strong (immediate)	"Content outside the agreed scope was detected."

When a warning is triggered, POST it to Person 1 and log it in `warning_history`.

---

5. Warning Format — Sent to Person 1

POST `http://localhost:8000/warnings/log`

```json
{
  "barter_id": 1,
  "severity": "mild",
  "reason": "Session appears to be drifting off topic",
  "window_ids": "3,4",
  "timestamp": "2026-03-12T10:23:45"
}
```

---

6. Session Termination Signal — Sent to Person 1

Triggered when `consecutive_incorrect >= 5`.

POST `http://localhost:8000/session/{barter_id}/terminate`

```json
{ "reason": "Severe and persistent topic drift detected" }
```

After sending, set `terminated = true` and ignore all further window results for this session.

---

7. Session End Endpoint

POST `/session/{barter_id}/end`

Called by Person 3 after flushing the final window. Build the drift summary and POST it to Person 1, then clean up all in-memory state for this session.

---

8. Drift Summary Format — Sent to Person 1

POST `http://localhost:8000/session/{barter_id}/drift-summary`


```json
{
  "barter_id": 1,
  "total_windows": 12,
  "incorrect_windows": 5,
  "percent_incorrect": 41.7,
  "max_consecutive_incorrect": 5,
  "warning_count": 3,
  "warnings": [
    {
      "severity": "mild",
      "reason": "Session appears to be drifting off topic",
      "window_ids": "3,4",
      "timestamp": "2026-03-12T10:23:45"
    },
    {
      "severity": "strong",
      "reason": "Session has consistently drifted from the agreed topic",
      "window_ids": "5,6,7",
      "timestamp": "2026-03-12T10:26:10"
    }
  ],
  "terminated_early": false
}
```

Interface Contracts

You Receive (inbound)

From	Endpoint	What
Person 1	`POST /session/{barter_id}/init`	Session start signal
Person 3	`POST /window/result`	Per-window classification
Person 3	`POST /session/{barter_id}/end`	Session end signal

You Call (outbound)

Target	When	Endpoint
Person 1	On warning trigger	`POST http://localhost:8000/warnings/log`
Person 1	On severe drift	`POST http://localhost:8000/session/{barter_id}/terminate`
Person 1	On session end	`POST http://localhost:8000/session/{barter_id}/drift-summary`

What You Do NOT Build

- STT or audio processing → Person 2
- Sentence-BERT or similarity scoring → Person 3
- Sliding window assembly → Person 3
- QA verdict logic → Person 1
- Trust score updates → Person 1
- Frontend or WebSocket broadcast to browser → Person 1

Libraries

```
fastapi, uvicorn, httpx
```

No ML libraries needed. This service is pure logic.

Definition of Done

- [ ] Service runs on port 8003
- [ ] Session init creates correct empty state
- [ ] Counters update correctly on each window (increment and reset)
- [ ] Correct severity triggered at consecutive counts 2, 3–4, 5+
- [ ] Out-of-scope windows trigger strong warning immediately
- [ ] Warning POSTed to Person 1 within 1 second of decision
- [ ] Termination signal sent to Person 1 when consecutive reaches 5+
- [ ] No further windows processed after termination
- [ ] Drift summary is accurate (correct totals, percentages, warning history)
- [ ] Drift summary POSTed to Person 1 before state is cleaned up
- [ ] State cleaned up after session ends